

Steven Davis

04/25/2026

CS 470 Final Reflection

Presentation Link: <https://youtu.be/bJPav23EOpc>

Experiences and Strengths

Throughout CS 470, I developed a strong understanding of cloud development by building and deploying a full-stack web application using modern tools and services. This course helped me move beyond basic development into cloud-based architecture, which is essential for my professional goals in software development. Learning how to integrate frontend and backend systems with cloud services such as APIs, databases, and storage has made me a more well-rounded and marketable candidate.

Some of the most important skills I learned include containerization with Docker, managing multi-container environments with Docker Compose, and migrating applications to a serverless cloud environment. I also gained experience working with AWS services such as Lambda, API Gateway, and S3, which helped me understand how scalable applications are built and maintained in the cloud. Additionally, I improved my understanding of API development, database integration, and testing, which are critical skills in modern software engineering.

My strengths as a software developer include problem-solving, adaptability, and the ability to learn new technologies quickly. Throughout this course, I was able to take complex cloud concepts and apply them in building a working application. I also demonstrated the ability to troubleshoot issues, integrate multiple technologies, and follow structured development processes. Based on these experiences, I feel prepared to take on roles such as junior software developer, cloud developer, or backend developer, where I can continue building scalable, secure applications.

Planning for Growth

As applications grow, it is important to design systems that can scale efficiently while maintaining performance and cost control. One of the most effective ways to achieve this is by using microservices and serverless architecture. Microservices allow an application to be broken into smaller, independent services that can be developed, deployed, and scaled individually. This improves flexibility and makes it easier to manage updates without affecting the entire system. Serverless architecture, on the other hand, removes the need to manage infrastructure and automatically scales based on demand, which is ideal for handling unpredictable workloads.

To handle scale and error management, I would design the system to use automatic scaling features provided by cloud services. For example, AWS Lambda can scale based on incoming requests, while API Gateway can manage traffic and route requests efficiently. Error handling would include implementing logging, monitoring, and retry mechanisms to detect and address failures quickly. Tools such as cloud monitoring services would help identify performance issues and maintain system reliability.

Predicting cloud costs is an important part of planning for growth. Serverless services use a pay-for-use model, which means costs are based on the number of requests and execution time. This makes serverless cost-efficient for applications with variable traffic, but can become unpredictable at scale. Containers, on the other hand, tend to have more predictable costs because they run on allocated resources, but they may result in paying for unused capacity. In general, serverless is more cost-efficient for smaller or unpredictable workloads, while containers provide more cost predictability for steady, high-traffic applications.

There are several pros and cons to consider when planning for expansion. Serverless architecture reduces operational overhead and scales automatically, but it can introduce challenges such as cold-start latency and reduced control over the environment. Containers provide more control and consistency across environments but require more management and maintenance. The decision between these approaches depends on the specific needs of the application, including performance requirements, traffic patterns, and cost considerations.

Elasticity and the pay-for-service model play a major role in decision-making for future growth. Elasticity allows applications to automatically scale up or down based on demand, ensuring performance while avoiding unnecessary costs. The pay-for-service model ensures that organizations only pay for the resources they use, which is especially beneficial for startups or applications with fluctuating traffic. Together, these concepts enable developers to design systems that are both efficient and scalable, supporting long-term growth without excessive resource usage.

CS 470 has strengthened my ability to design and develop cloud-based applications while preparing me for real-world software development roles. By combining my technical skills with an understanding of scalability, cost management, and cloud architecture, I am better equipped to build applications that can grow and adapt over time. This experience has not only improved my development skills but also given me the confidence to work with modern cloud technologies in a professional environment.